

OpenClaw Agentic Job Automation

Final agreed architecture and implementation snapshot

Final architecture decision

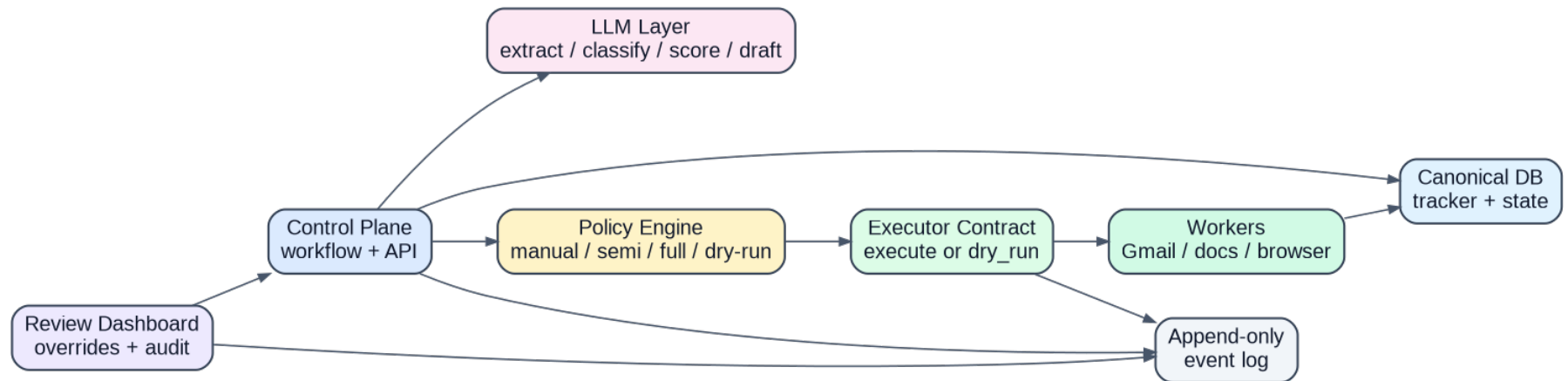
- Build a governed workflow platform, not a monolithic AI bot.
- Workflow owns state. Database owns truth. Policy engine owns permission.
- LLM is constrained to extraction, classification, scoring support, and drafting.
- Workers execute approved actions only through a shared executor contract.
- Dry-run is a platform capability from the start, not a late add-on.
- Semi-auto and full-auto are policy modes on the same workflow.

Final agreed build order

1. Canonical data model and tracker
2. Application state machine
3. Append-only event log
4. Policy engine and automation modes
5. Executor contract with execute and dry_run support
6. Manual job intake
7. Job parsing and basic scoring
8. Application packet generation
9. Review dashboard
10. Recruiter email classification and draft replies
11. Browser form-filling worker
12. Narrow full-auto rules with allowlists, rate limits, and kill switch

1. Final architecture

Control plane, policy engine, LLM layer, executor contract, workers, dashboard, canonical DB, and event log.



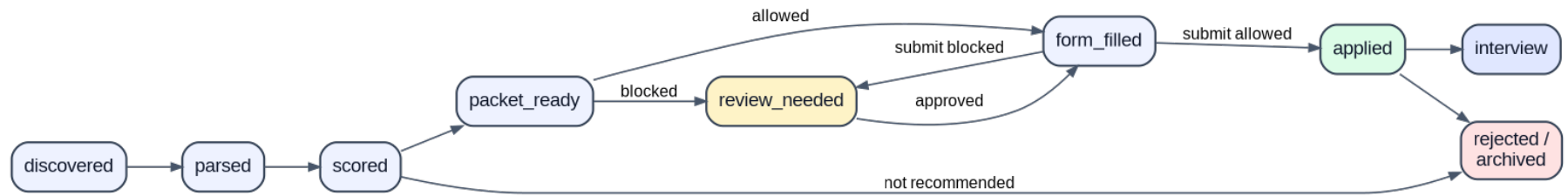
2. Core workflow

Policy decision is logged before execution. Execution result is logged after execution.



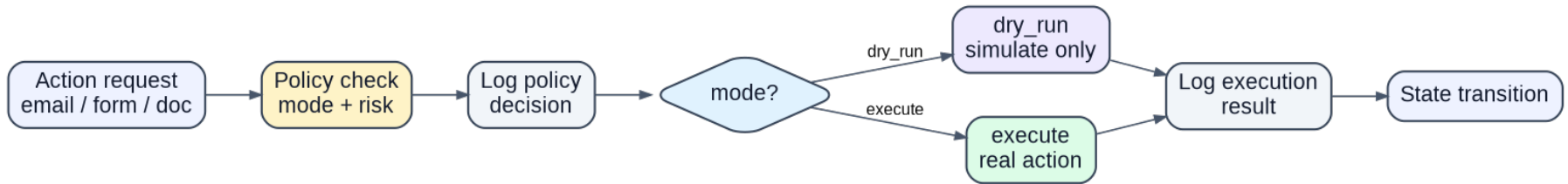
3. Application state machine

The tracker owns the current state. The event log records all transitions and decisions.



4. Executor contract with dry-run

All outbound-capable workers support execute and dry_run through the same contract.



5. Final build order

Build the safety and workflow foundation before Gmail, browser automation, or full-auto submit.



6. Quick implementation plan

Milestone	Name	Outcome
M1	Control plane foundation	Data model, tracker, state machine, event log, policy modes
M2	Executor foundation	Shared executor interface with execute and dry_run support
M3	Manual intake MVP	Paste job, normalize record, dedupe, save to tracker
M4	Scoring MVP	Parse job, score fit, explain recommendation, skip reasons
M5	Packet MVP	Draft CV bullets, cover note, screening answers, store packet
M6	Review dashboard	Review queue, state transitions, audit visibility, overrides
M7	Recruiter email MVP	Classify emails, update tracker, draft replies through policy gate
M8	Browser worker MVP	Fill forms and upload CV; pause or dry-run before submit
M9	Full-auto narrow mode	Allowlists, rate limits, kill switch, fallback to semi-auto

Full-auto guardrails v1

- Trusted domains only
- Known ATS templates only
- No custom questions
- No salary/legal/disclosure questions
- High fit threshold
- Per-day and per-company caps
- Global kill switch
- Fallback to semi-auto on low confidence, captcha, unknown field, login issue, or conflicting data

Recommended stack

- Frontend: Next.js + React + Tailwind
- Backend: FastAPI
- Workflow: custom state machine first; LangGraph later if useful
- DB: PostgreSQL; pgvector later
- Queue: Redis + RQ/Celery
- Automation: Playwright
- Docs: docxtpl / python-docx + PDF export
- Email: Gmail API
- Deploy: Docker Compose first

First concrete target

- Build tracker + state machine + event log + policy engine + dry-run executor stub before any Gmail, browser, or LLM integration.
- Prove the system can create an application record, move through states, log every transition, evaluate policy, and simulate actions safely.
- After that, plug in job parsing, packet generation, email, and browser automation as replaceable workers.